Assembly for Reverse Engineering

Bit Operations & Arithmetic Instructions

Barak Gonen

Bit Operations

- AND
- OR
- XOR

```
mov     eax, 0xFFFFFFFF
and     ax, 0x00FF
xor     al, 0xFF
or      ax, 0xFFFF
```

- SHR
- SHL

```
mov     eax, 0x0000FF0F
shl     eax, 4
shr     eax, 1
shl     eax, 32
```

Riddle

How can we tell if a number is even / odd?

… Can be divided by 4 without remainder?

Riddle

How can we set the leftmost bit in EAX to 1?

Arithmetic Instructions

- INC
- DEC
- ADD
- SUB
- MUL
- DIV

```
mov     eax, 0x15
add     eax, 0x11
inc     eax
mov     ebx, 0x20
sub     eax, ebx
dec     eax
mov     eax, 0xFFFFFFFF
add     eax, 2
```

MUL

```
mov     al, 0x61
mov     bl, 3
mul     bl   ; al*bl -> ax

mov     ax, 0x1234
mov     bx, 0x5678
mul     bx   ; ax*bx -> dx:ax
             ; dx=0x626, ax=0x60

mov     eax, 0xABCD1234
mov     ebx, 0x10
mul     ebx  ; eax*abx -> edx:eax
             ; edx=0xA eax=0xBCD12340
```

DIV

```
mov     ax, 0x0F02
mov     bl, 0x10
div     bl   ; ax/bl
             ; al - mana (0xF0) , ah - sheerit (2)

mov     edx, 0
mov     eax, 0x0000F002
mov     bx, 0x10
div     bx   ; dx:ax/bx
             ; ax - mana (0xF00), dx - sheerit (2)

mov     edx, 0x0000000F
mov     eax, 0x00000002
mov     ebx, 0x10
div     ebx  ; edx:eax/ebx
             ; eax- mana (0xF000000), edx - sheerit (2)
```